```
### Question 1
def sum_product(*numbers, in_mode=0):
    if len(numbers) == 0:
        return None, None

    if in_mode == 0:
        # tminimum and total sum
        smallest = numbers[0]
        total = 0
        for n in numbers:
            if n < smallest:
                smallest = n
            total += n
        return smallest, total
    else:
        # te maximum and total product
        largest = numbers[0]
        result = 1
        for n in numbers:
            if n > largest:
                largest = n
            result *= n
        return largest, result

print(sum_product(1, 2, 3, 4))
print(sum_product(1, 2, 3, 4, in_mode=1))
print(sum_product())
```

```
### Question 2
def find_gpa(*args):
    # if numb of inputs is even
    if len(args) % 2 != 0:
        print("Error: Please enter credit and grade in pairs.")
        return -1

    total_credits = 0
    total_points = 0

    for i in range(0, len(args), 2):
        credit = args[i]
        grade = args[i + 1]
        total_credits += credit
        total_points += credit * grade

    gpa = total_points / total_credits
    return round(gpa, 2)

#test cases
print(find_gpa(3, 3.20, 4, 2.80, 2, 3.70))
print(find_gpa(3, 3.20, 4))
```

```
### Question 3
import random
def tortoise_move(pos):
    i = random.randint(1, 120)
    #print("tortoise roll:i)#used for test
    if 1 <= i <= 36:            #fast plod
        pos += 3
    elif 37 <= i <= 60: #slip
        pos -= 6
    else:  #slowplod
        pos += 1

    if pos < 1:
        pos = 1
    return pos
```

```
def hare_move(pos):
    i = random.randint(1, 120)
    #print("hare roll:", i)  # testin values
    if 1 <= i <= 20:      #sleep, no move
        pass
    elif 21 <= i <= 30:  #big hop
        pos += 8
    elif 31 <= i <= 40: #big slip
        pos -= 10
    elif 41 <= i <= 88: #small hop
        pos += 1
    else:   #small slip
        pos -= 2

    if pos < 1:
        pos = 1
    return pos

def display_course(t_pos, h_pos):
    track = ["-"] * 60

    if t_pos == h_pos:
        track[t_pos - 1] = "OUCH!"
    else:
        if t_pos <= 60:
            track[t_pos - 1] = "T"
        if h_pos <= 60:
            track[h_pos - 1] = "H"

    #for displayin race
    print("".join(str(s).ljust(1) for s in track))

def run_race():
    t = 1 #tortoise pos
    h = 1 #hare pos
    print("BAnnG! AND THEY'RE OFF!!")
    while t < 60 and h < 60:
        t = tortoise_move(t)
        h = hare_move(h)
        display_course(t, h)
        if t >= 60 and h >= 60:
            print("It's a TIE.")
            return "Tie"
        elif t >= 60:
            print("TORTOISE WINS!!! YAY!!!")
            return "Tortoise"
        elif h >= 60:
            print("HARE WINS. YUCH.")
            return "Hare"
def simulate_races():
    t_w = 0
    h_w = 0
    ties = 0

    for x in range(50): #simlate 50 times
        #if it loops
        result = run_race()
        if result == "Tortoise":
            t_w += 1
        elif result == "Hare":
            h_w += 1
        else:
            ties += 1

        print("-" * 70)

    print("FINAL RESULTS AFTER 50 RACES")
    print("Tortoise wins:", t_w)
    print("Hare wins:", h_w)
    print("Ties:", ties)

#running sim
simulate_races()
```

```
### Question 4
def sieve_primes(limit):
    #step a: start with all numbers marked true (maybe prime)
    primes = [True] * limit

    #0 and 1 are not prim
    primes[0] = False
    primes[1] = False

    #step b/c: mark non-primes (not prime ones)
    p = 2
    while p * p < limit:
        if primes[p]:
            # mark all multples of p as False
            for i in range(p * 2, limit, p):
                primes[i] = False
        p += 1  # go to next
    return primes

#check primes under 25000 (limit)
limit = 25000
prime_flags = sieve_primes(limit)
#print all the prime numz under 25000
for i in range(len(prime_flags)):
    if prime_flags[i]:
        print(i)




### Question 5
def tennis_score(score, player1, player2):
    points_text = ["Love", "Fifteen", "Thirty", "Forty"]

    #split and convert scres
    p1, p2 = score.split(":")
    p1 = int(p1)
    p2 = int(p2)

    #deuce
    if p1 >= 3 and p2 >= 3:
        if p1 == p2:
            return "Deuce"
        elif abs(p1 - p2) == 1:
            return "Advantage " + (player1 if p1 > p2 else player2)
        elif abs(p1 - p2) >= 2:
            return "Game " + (player1 if p1 > p2 else player2)

    #agme win without deuce
    if p1 >= 4 and p1 - p2 >= 2:
        return "Game " + player1
    if p2 >= 4 and p2 - p1 >= 2:
        return "Game " + player2

    #normal score
    return f"{points_text[p1]} {points_text[p2]}"

print(tennis_score("1:0", "Alex", "Sarah"))
print(tennis_score("3:3", "Alex", "Sarah"))
print(tennis_score("2:2", "Alex", "Sarah"))
print(tennis_score("4:3", "Alex", "Sarah"))
print(tennis_score("6:4", "Alex", "Sarah"))



### Question 6
import os

# make the data.txt if it aint there yet
def create_sample_file():
    if not os.path.exists("data.txt"):
        with open("data.txt", "w") as f:
```

```
            f.write("SMP613NZBEXAR: Samsung Galaxy Tab S6 Lite 64GB, $344.99, 10\n")
            f.write("APL555AIRPODS: Apple AirPods Pro 2nd Gen, $249.99, 8\n")
            f.write("BOK897234BOOK: The Alchemist by Paulo Coelho, $12.50, 25\n")
            f.write("LGTV2023123: LG 55\" OLED TV, $999.99, 3\n")

# load all the stuff from the file
def load_products():
    stuff = []
    try:
        with open("data.txt", "r") as f:
            for line in f:
                try:
                    upc, rest = line.strip().split(":")
                    name, price, qty = [x.strip() for x in rest.split(",")]
                    stuff.append({
                        "upc": upc,
                        "name": name,
                        "price": float(price.strip("$")),
                        "qty": int(qty)
                    })
                except:
                    print("oops bad line, skipped it")
    except:
        print("uhh file not found?")
    return stuff

# save back the stuff
def save_products(stuff):
    with open("data.txt", "w") as f:
        for p in stuff:
            f.write(f"{p['upc']}: {p['name']}, ${p['price']:.2f}, {p['qty']}\n")

# 1 - add new item
def add_product():
    upc = input("UPC? ")
    name = input("Name? ")
    try:
        price = float(input("Price? "))
        qty = int(input("Qty? "))
    except:
        print("nah thats not right")
        return
    with open("data.txt", "a") as f:
        f.write(f"{upc}: {name}, ${price:.2f}, {qty}\n")
    print("added it!")

# 2 - change item quantity
def update_quantity():
    upc = input("UPC to update? ")
    try:
        new_qty = int(input("New quantity? "))
    except:
        print("not a number bro")
        return
    stuff = load_products()
    found = False
    for p in stuff:
        if p["upc"] == upc:
            p["qty"] = new_qty
            found = True
            break
    if found:
        save_products(stuff)
        print("updated yay")
    else:
        print("couldnt find it")

# 3 - total value of everything
def total_value():
    stuff = load_products()
    total = sum(p["price"] * p["qty"] for p in stuff)
    print(f"Total value: ${total:.2f}")

# 4 - most expensive item
def most_expensive():
```

```
    stuff = load_products()
    if not stuff:
        print("nothing here")
        return
    maxp = max(stuff, key=lambda p: p["price"])
    print(f"{maxp['name']} - ${maxp['price']:.2f}")

# 5 - show item with lowest qty
def lowest_qty():
    stuff = load_products()
    if not stuff:
        print("file empty :(")
        return
    minp = min(stuff, key=lambda p: p["qty"])
    print(f"{minp['name']} - only {minp['qty']} left")

# 6 - show stuff with low stock
def below_qty():
    try:
        n = int(input("Show items below what qty? "))
    except:
        print("enter a real number bruh")
        return
    stuff = load_products()
    found = False
    for p in stuff:
        if p["qty"] < n:
            print(f"{p['name']} - {p['qty']}")
            found = True
    if not found:
        print("all good")

# 7 - sort stuff by price low to high
def sort_by_price():
    stuff = load_products()
    stuff.sort(key=lambda p: p["price"])
    save_products(stuff)
    print("sorted them!")

# menu menu menu
def menu():
    create_sample_file()
    while True:
        print("\n--- INVENTORY MENU ---")
        print("1. Add product")
        print("2. Update quantity")
        print("3. Show total value")
        print("4. Show most expensive")
        print("5. Show lowest quantity")
        print("6. Show all below a quantity")
        print("7. Sort by price")
        print("8. Exit")
        choice = input("Pick 1-8: ")

        if choice == "1":
            add_product()
        elif choice == "2":
            update_quantity()
        elif choice == "3":
            total_value()
        elif choice == "4":
            most_expensive()
        elif choice == "5":
            lowest_qty()
        elif choice == "6":
            below_qty()
        elif choice == "7":
            sort_by_price()
        elif choice == "8":
            print("bye bye")
            break
        else:
            print("wrong number try again")

# run ittt
    ``
```

```
menu()
```

+ Code + Text